

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en Computación



LABORATORIO S11

Mexi Banco: una verdad, varias copias

Bitácora de implementación

DATOS ACADÉMICOS

Alumna:	Estrella de MarIa Mendoza Camacho
Número de cuenta:	321132927
Asignatura:	Sistemas Distribuidos
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre:	2027-1

1. Preparación del entorno

1.1 Levantamiento de los servicios

Se utilizó la rama base correspondiente a la práctica y se inició la infraestructura mediante Docker Compose.

El proyecto cuenta con varios servicios, entre ellos el gateway, el directorio de servicios, tres instancias de Cuenta, Transferencia y Notificación. También se incluyen PostgreSQL, Redis, MongoDB y RabbitMQ.

El objetivo de este primer paso fue comprobar que los contenedores estuvieran disponibles antes de realizar cambios en las aplicaciones.

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE         CREATED              STATUS                        PORTS
mexi-banco-v06-cuenta-1       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          3 minutes ago        Up 3 minutes (healthy)        8080/tcp
mexi-banco-v06-cuenta-2       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          3 minutes ago        Up 3 minutes (healthy)        8080/tcp
mexi-banco-v06-cuenta-3       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          3 minutes ago        Up 3 minutes (healthy)        8080/tcp
mexi-banco-v06-db-1           postgres:17-alpine             "docker-entrypoint.s…"   db              3 minutes ago        Up 3 minutes (healthy)        5432/tcp
mexi-banco-v06-discovery-1    mexi-banco-v06-discovery       "java -jar app.jar"      discovery       3 minutes ago        Up 3 minutes (healthy)        0.0.0.0:8761->8761/tcp, [
::]:8761->8761/tcp
mexi-banco-v06-gateway-1      mexi-banco-v06-gateway         "java -jar app.jar"      gateway         3 minutes ago        Up 3 minutes (healthy)        0.0.0.0:8080->8080/tcp, [
::]:8080->8080/tcp
mexi-banco-v06-mongo-1        mongo:8.0                      "docker-entrypoint.s…"   mongo           About a minute ago   Up About a minute (healthy)   127.0.0.1:27017->27017/tc
p
mexi-banco-v06-movimiento-1   mexi-banco-v06-movimiento       "java -jar app.jar"      movimiento      3 minutes ago        Up 3 minutes (healthy)        8080/tcp
mexi-banco-v06-notificacion-1 mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    3 minutes ago        Up 58 seconds (healthy)       8080/tcp
mexi-banco-v06-rabbitmq-1     rabbitmq:4-management          "docker-entrypoint.s…"   rabbitmq        3 minutes ago        Up About a minute (healthy)   0.0.0.0:5672->5672/tcp, [
::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
mexi-banco-v06-redis-1        redis:8-alpine                 "docker-entrypoint.s…"   redis           3 minutes ago        Up 3 minutes (healthy)        127.0.0.1:6379->6379/tcp
mexi-banco-v06-spei-1         mexi-banco-v06-spei            "java -jar app.jar"      spei            3 minutes ago        Up 3 minutes (healthy)        8080/tcp
mexi-banco-v06-transferencia-1 mexi-banco-v06-transferencia  "java -jar app.jar"      transferencia   3 minutes ago        Up 58 seconds (healthy)       8080/tcp
PS C:\Users\Billie Jean\mexi-banco-s11>
```

Figura 1: Contenedores de Mexi Banco iniciados y en estado saludable.

1.2 Verificación de Redis y MongoDB

Para comprobar el funcionamiento de Redis se ejecutó el comando `redis-cli ping` , el cual respondió `PONG` .

Posteriormente, se consultó la cantidad de documentos existentes en la colección de avisos de MongoDB. El resultado inicial fue cero, por lo que se contaba con una colección vacía antes de las pruebas.

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T redis redis-cli ping
PONG
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
0
PS C:\Users\Billie Jean\mexi-banco-s11>
```

Figura 2: Redis responde PONG y MongoDB presenta inicialmente cero avisos.

1.3 Consulta de las cuentas existentes

Al intentar crear las cuentas de Ana, Beto y Carla, el servidor devolvió el código HTTP 409, indicando que las cuentas ya existían.

Por esta razón, se consultaron los saldos existentes en lugar de eliminar la información de PostgreSQL.

Los resultados fueron los siguientes:

```
PS C:\Users\Billie Jean\mexi-banco-s11> foreach ($clabe in $clabes) {
>> Invoke-RestMethod `
>> -Uri "http://localhost:8080/api/cuentas/$clabe" |
>> Format-List clabe, titular, saldo
>> }


clabe   : 002180000000000001
titular : Ana
saldo   : 850.00




clabe   : 002180000000000002
titular : Beto
saldo   : 650.00




clabe   : 002180000000000003
titular : Carla
saldo   : 300.00



PS C:\Users\Billie Jean\mexi-banco-s11>
```

Figura 3: Consulta de las tres cuentas existentes en PostgreSQL.

1.4 Primera transferencia

Se realizó una transferencia de $200.00 desde Ana hacia Beto utilizando el endpoint de transferencias del gateway.

La operación se completó correctamente, con identificador 14 y estado `COMPLETADA` .

Como resultado, el saldo de Ana pasó a $650.00 y el de Beto a $850.00.

```
PS C:\Users\Billie Jean\mexi-banco-s11> $datos = @{
>> claveOrigen = "0021800000000000001"
>> claveDestino = "0021800000000000002"
>> monto = 200
>> } | ConvertTo-Json
PS C:\Users\Billie Jean\mexi-banco-s11> Invoke-RestMethod `
>> -Method Post `
>> -Uri "http://localhost:8080/api/transferencias" `
>> -ContentType "application/json" `
>> -Body $datos |
>> Format-List id, estado, monto, eventoId


id       : 14
estado   : COMPLETADA
monto    : 200
eventoId : e415e357-d9c0-4cd5-bc89-db3353c10b28
```

Figura 4: Primera transferencia completada desde Ana hacia Beto.

2. Consultas antes de utilizar Redis

2.1 Lecturas desde PostgreSQL

Antes de activar la caché se realizaron diez consultas consecutivas al saldo de Ana.

Todas devolvieron el mismo saldo de $650.00, pero con valores diferentes en el campo `leidaEn` .

Además, los registros de las instancias de Cuenta mostraron mensajes de lectura desde PostgreSQL.

Esto permitió establecer el comportamiento inicial del sistema, donde las consultas se resolvían directamente mediante la base de datos principal.

```
PS C:\Users\Billie Jean\mexi-banco-s11> 1..10 | ForEach-Object {
>> Invoke-RestMethod `
>> -Uri "http://localhost:8080/api/cuentas/00218000000000001" |
>> Select-Object clabe, saldo, leidaEn
>> }

clabe              saldo leidaEn
-----              ----- -------
00218000000000001 650.00 2026-11-07T00:15:12.234990568Z
00218000000000001 650.00 2026-11-07T00:15:12.319094121Z
00218000000000001 650.00 2026-11-07T00:15:12.341146021Z
00218000000000001 650.00 2026-11-07T00:15:12.357927547Z
00218000000000001 650.00 2026-11-07T00:15:12.376467665Z
00218000000000001 650.00 2026-11-07T00:15:12.420640372Z
00218000000000001 650.00 2026-11-07T00:15:12.434581889Z
00218000000000001 650.00 2026-11-07T00:15:12.449004959Z
00218000000000001 650.00 2026-11-07T00:15:12.464240417Z
00218000000000001 650.00 2026-11-07T00:15:12.479444671Z


PS C:\Users\Billie Jean\mexi-banco-s11> docker compose logs --since=2m cuenta |
>> Select-String "leida de Postgres"

cuenta-3  | 2026-11-07T00:13:34.522Z  INFO 1 --- [cuenta] [nio-8080-exec-3]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000002 leida de Postgres
cuenta-2  | 2026-11-07T00:13:34.508Z  INFO 1 --- [cuenta] [nio-8080-exec-9]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-2  | 2026-11-07T00:15:12.334Z  INFO 1 --- [cuenta] [nio-8080-exec-2]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-2  | 2026-11-07T00:15:12.417Z  INFO 1 --- [cuenta] [nio-8080-exec-3]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-2  | 2026-11-07T00:15:12.461Z  INFO 1 --- [cuenta] [nio-8080-exec-4]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-1  | 2026-11-07T00:15:12.229Z  INFO 1 --- [cuenta] [nio-8080-exec-6]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-1  | 2026-11-07T00:15:12.353Z  INFO 1 --- [cuenta] [nio-8080-exec-5]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-1  | 2026-11-07T00:15:12.431Z  INFO 1 --- [cuenta] [nio-8080-exec-7]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-1  | 2026-11-07T00:15:12.476Z  INFO 1 --- [cuenta] [nio-8080-exec-9]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-3  | 2026-11-07T00:15:12.314Z  INFO 1 --- [cuenta] [nio-8080-exec-9]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-3  | 2026-11-07T00:15:12.372Z  INFO 1 --- [cuenta] [io-8080-exec-10]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
cuenta-3  | 2026-11-07T00:15:12.446Z  INFO 1 --- [cuenta] [nio-8080-exec-1]
mx.mexibanco.cuenta.ConsultaDeSaldo      : [consulta] 00218000000000001 leida de Postgres
```

Figura 5: Diez consultas consecutivas y registros de lectura en PostgreSQL.

2.2 Estado inicial de Redis

Después de las consultas se revisaron los contadores de Redis:

- Aciertos: 0.
- Fallos: 0.

Ambos valores confirmaron que todavía no se estaban realizando búsquedas mediante la caché.

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T redis redis-cli INFO stats |
>> Select-String "keyspace_hits|keyspace_misses"

keyspace_hits:0
keyspace_misses:0
```

Figura 6: Contadores iniciales de Redis sin aciertos ni fallos.

3. Implementación de la caché Redis

3.1 Modificaciones en Cuenta

Se modificó la configuración de caché del servicio Cuenta para utilizar Redis como almacenamiento temporal de los saldos.

Los principales cambios fueron:

- En `ConfiguracionCache.java` se habilitó el sistema de caché de Spring y se estableció un TTL de 30 segundos.
- En `ConsultaDeSaldo.java` se utilizó `@Cacheable` para almacenar los resultados de las consultas por CLABE.
- En `CuentaService.java` se utilizó `@CacheEvict` para eliminar los saldos almacenados cuando una cuenta recibe un cargo o un abono.

La finalidad fue evitar consultas repetidas a PostgreSQL sin utilizar datos de Redis para autorizar movimientos bancarios.

La configuración también incorporó `transactionAware()` , para coordinar las operaciones de caché con la confirmación de las transacciones.

3.2 Compilación y reinicio

Se reconstruyeron los servicios Cuenta y Notificación, y se comprobó que las tres instancias de Cuenta y Notificación se encontraban en estado `healthy` .

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose ps cuenta notificacion
NAME                            IMAGE                          COMMAND              SERVICE        CREATED              STATUS                          PORTS
mexi-banco-v06-cuenta-1         mexi-banco-v06-cuenta          "java -jar app.jar"  cuenta         About a minute ago   Up About a minute (healthy)   8080/tcp
mexi-banco-v06-cuenta-2         mexi-banco-v06-cuenta          "java -jar app.jar"  cuenta         About a minute ago   Up About a minute (healthy)   8080/tcp
mexi-banco-v06-cuenta-3         mexi-banco-v06-cuenta          "java -jar app.jar"  cuenta         About a minute ago   Up About a minute (healthy)   8080/tcp
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion    "java -jar app.jar"  notificacion   About a minute ago   Up About a minute (healthy)   8080/tcp
PS C:\Users\Billie Jean\mexi-banco-s11>
```

Figura 7: Tres instancias de Cuenta y Notificación funcionando correctamente.

3.3 Almacenamiento y caducidad

Después de habilitar Redis se consultó la cuenta de Ana para generar una entrada en caché.

Se verificó que Redis almacenara un documento JSON con la CLABE, el titular, el saldo y la fecha de lectura.

También se consultó el tiempo de vida de la clave, obteniendo un TTL restante de 15 segundos.

Esto confirmó que los datos no se almacenan indefinidamente.

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T redis redis-cli TTL cuentas::00218000000000001
15
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T redis redis-cli GET cuentas::00218000000000001
{"clabe":"00218000000000001","titular":"Ana","saldo":650.00,"leidaEn":"2026-11-07T06:37:19.169956386Z"}
PS C:\Users\Billie Jean\mexi-banco-s11>
```

Figura 8: Saldo de Ana almacenado en Redis con un TTL restante de 15 segundos.

3.4 Actualización del saldo

Se realizó una transferencia de $10.00 desde Ana hacia Beto.

Después de la operación se consultó nuevamente el saldo de Ana, obteniendo $640.00.

El campo `leidaEn` también mostró una fecha posterior a la lectura previamente almacenada.

Este resultado fue consistente con la actualización de la información consultada después de una transferencia.

Sin embargo, la comparación del saldo por sí sola no permite distinguir entre una invalidación y una expiración natural de la clave. Para demostrar el borrado inmediato se requiere observar la eliminación antes de que termine el TTL.

```
PS C:\Users\Billie Jean\mexi-banco-s11> Invoke-RestMethod -Uri "http://localhost:8080/
01" |
>>     Select-Object clabe, saldo, leidaEn

clabe               saldo leidaEn
-----               ----- -------
00218000000000001 640.00 2026-11-07T06:38:54.797925947Z
```

Figura 9: Consulta del saldo actualizado de Ana después de una transferencia.

4. Medición de aciertos en Redis

4.1 Escenario A: consultas sin transferencias

Se eliminaron las entradas previas de Redis y se reiniciaron sus estadísticas.

Posteriormente, se realizaron 100 consultas consecutivas a la cuenta de Ana.

Los resultados fueron:

- Aciertos: 99.
- Fallos: 1.

El porcentaje de aciertos se calculó mediante:

$$Hitratio = \frac{Aciertos}{Aciertos + Fallos} \times 100$$

$$Hitratio_A = \frac{99}{99+1} \times 100 = 99\,\%$$

Esto indica que, después de la primera consulta, las siguientes pudieron aprovechar el dato guardado temporalmente en Redis.

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T redis redis-cli INFO stats |
>>     Select-String "keyspace_hits|keyspace_misses"

keyspace_hits:99
keyspace_misses:1
```

Figura 10: Redis registró 99 aciertos y un fallo durante 100 consultas.

4.2 Escenario B: consultas con transferencias

En el segundo escenario se repitieron 100 consultas, pero se realizó una transferencia de $1.00 de Ana a Beto cada diez consultas.

Cada transferencia modifica los saldos y puede provocar la invalidación de sus entradas en Redis.

Los contadores finales fueron:

- Aciertos: 100.
- Fallos: 20.

El porcentaje obtenido fue:

$$Hitratio_B = \frac{100}{100+20} \times 100 = 83.33\,\%$$

Los contadores incluyen las búsquedas registradas por Redis durante el escenario, por lo que el total puede ser mayor que las cien peticiones GET realizadas.

```
PS C:\Users\Billie Jean\mexi-banco-s11> docker compose exec -T redis redis-cli INFO stats |
>> Select-String "keyspace_hits|keyspace_misses"

keyspace_hits:100
keyspace_misses:20
```

Figura 11: Redis registró 100 aciertos y 20 fallos con transferencias.

4.3 Comparación de resultados

Cuadro 1: Comparación de los escenarios de Redis.

Indicador	Escenario A	Escenario B
Aciertos	99	100
Fallos	1	20
Hit ratio	99.00 %	83.33 %

> El porcentaje de aciertos disminuyó cuando se realizaron transferencias, porque los cambios en los saldos hacen necesario renovar los datos almacenados en caché.

5. Comparación de latencia

5.1 Prueba con Redis activado

Se midieron los tiempos de respuesta de 100 consultas al saldo de Ana.

Con Redis activado se obtuvieron los siguientes resultados:

- Promedio: 14.28 ms.
- Mediana: 13.09 ms.
- Percentil 95: 19.30 ms.

La mediana representa el valor central de los tiempos ordenados, mientras que el percentil 95 permite observar el comportamiento de la mayoría de las consultas, incluyendo las más lentas.

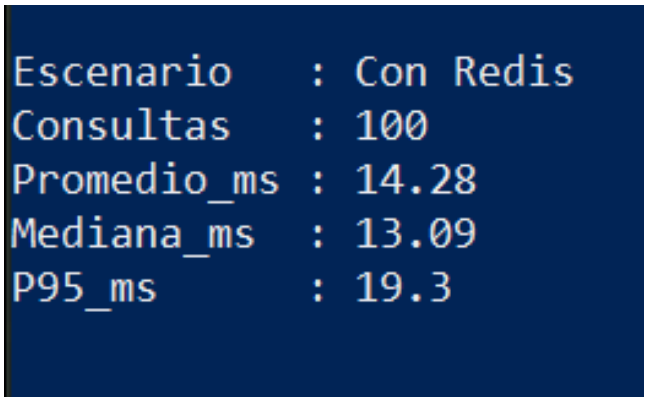


Figura 12: Medición de latencia en 100 consultas con Redis.

5.2 Prueba sin caché

Se desactivó temporalmente la caché y se repitieron 100 consultas utilizando el mismo endpoint.

Los resultados fueron:

- Promedio: 27.86 ms.
- Mediana: 20.93 ms.
- Percentil 95: 28.54 ms.

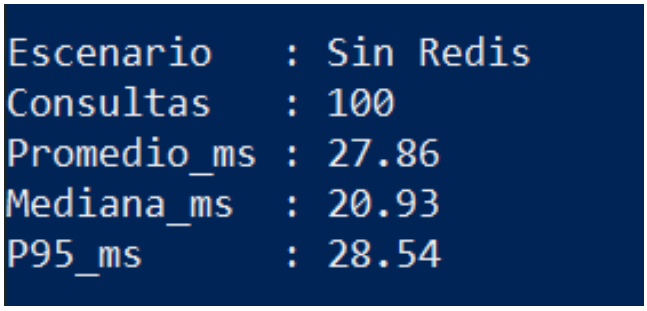


Figura 13: Medición de latencia en 100 consultas sin Redis.

5.3 Análisis del rendimiento

Cuadro 2: Comparación de latencia con y sin caché.

Métrica	Con Redis	Sin Redis
Promedio	14.28 ms	27.86 ms
Mediana	13.09 ms	20.93 ms
P95	19.30 ms	28.54 ms

La reducción porcentual del tiempo promedio se calculó como:

$$\textit{Reducción} = \frac{27.86 - 14.28}{27.86} \times 100$$

$$\textit{Reducción} = 48.74\,\%$$

> En las mediciones realizadas, Redis redujo aproximadamente 48.7 % el tiempo promedio de respuesta de las consultas.

6. Implementación de MongoDB

6.1 Almacenamiento de notificaciones

Se modificó el servicio Notificación para almacenar los avisos de transferencias en MongoDB.

En `BitacoraDeAvisos.java` se implementó la construcción de documentos, su almacenamiento, la consulta de los avisos recientes y la creación de un índice.

En `ConsumidorDeTransferencias.java` se conectó el procesamiento de eventos de RabbitMQ con el registro de los documentos.

Los avisos contienen campos comunes como el identificador del evento, la transferencia, la CLABE, el monto, el saldo y la fecha.

También almacenan información específica según el canal:

- SMS: teléfono y texto.
- CORREO: dirección, asunto y cuerpo.
- PUSH: token del dispositivo, título y cuerpo.

Los montos y saldos se almacenaron mediante `Decimal128`, con el propósito de conservar la precisión decimal requerida para valores monetarios.

6.2 Verificación de documentos

Al consultar MongoDB se encontraron 24 documentos generados por las transferencias realizadas durante las pruebas de Redis.

También se observaron documentos de tipo SMS y CORREO.

Los registros utilizaron un identificador compuesto por el identificador del evento y la CLABE del destinatario.

Además, se comprobó la existencia del índice `clabe_fecha`.

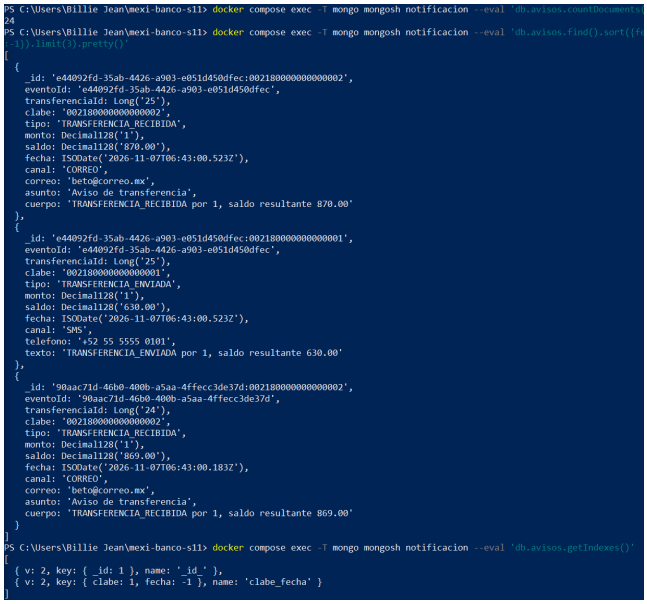


Figura 14: Avisos almacenados en MongoDB y existencia del índice compuesto.

6.3 Notificación PUSH

Para comprobar el tercer canal se realizó una transferencia de $1.00 desde Ana hacia Carla.

La operación se completó con identificador 26.

MongoDB generó un documento con el canal `PUSH`, el token del dispositivo, el título y el cuerpo del aviso.

El documento también mostró el saldo resultante de Carla, equivalente a $301.00.

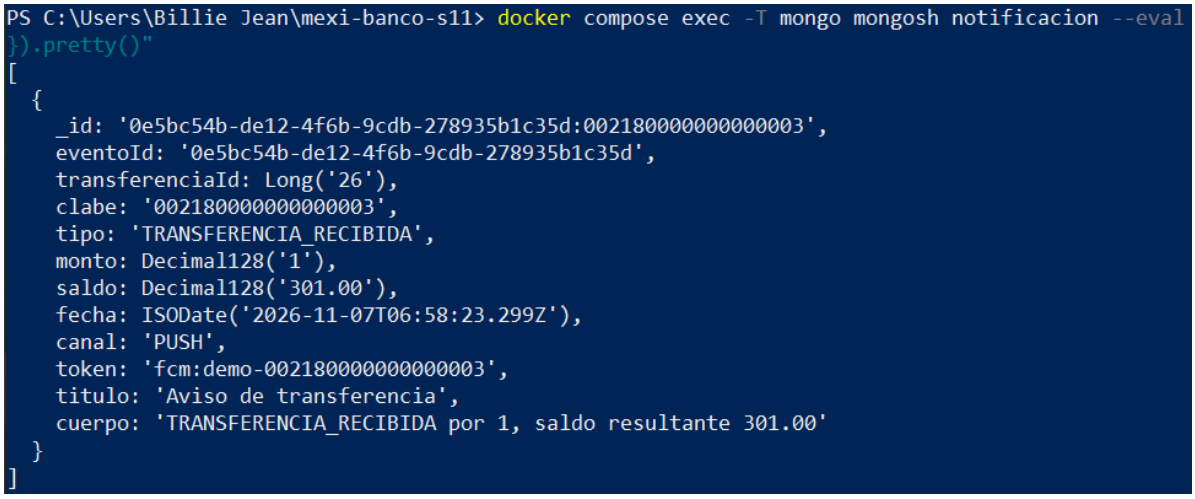


Figura 15: Documento de notificación PUSH generado para Carla.

7. Consulta de los últimos avisos

Se utilizó el endpoint de Notificación para consultar los avisos de Beto.

La respuesta de la API mostró una cantidad de diez registros.

Posteriormente, se ejecutó una consulta en MongoDB ordenada por fecha descendente y limitada a diez documentos.

Ambas consultas confirmaron el límite esperado.

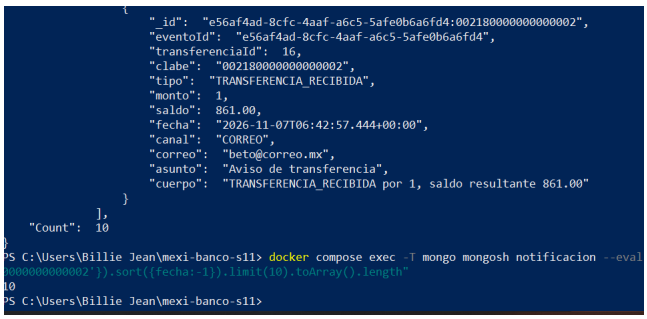


Figura 16: Consulta de los últimos diez avisos de Beto.

8. Evaluación del índice en MongoDB

Para comprobar el comportamiento del índice `clabe_fecha` se utilizó `explain()` con estadísticas de ejecución.

Primero se consultaron los últimos diez avisos de Beto utilizando el índice compuesto.

Posteriormente, se ejecutó la misma consulta forzando un recorrido de la colección, para comparar ambos planes.

Los resultados fueron:

Cuadro 3: Comparación de planes de consulta.

Indicador	Con índice	Sin índice
Plan	IXSCAN	COLLSCAN
Documentos devueltos	10	10
Documentos examinados	10	26
Ordenamiento adicional	No	Sí

La reducción de documentos examinados fue:

$$\frac{26 - 10}{26} \times 100 = 61.54\,\%$$

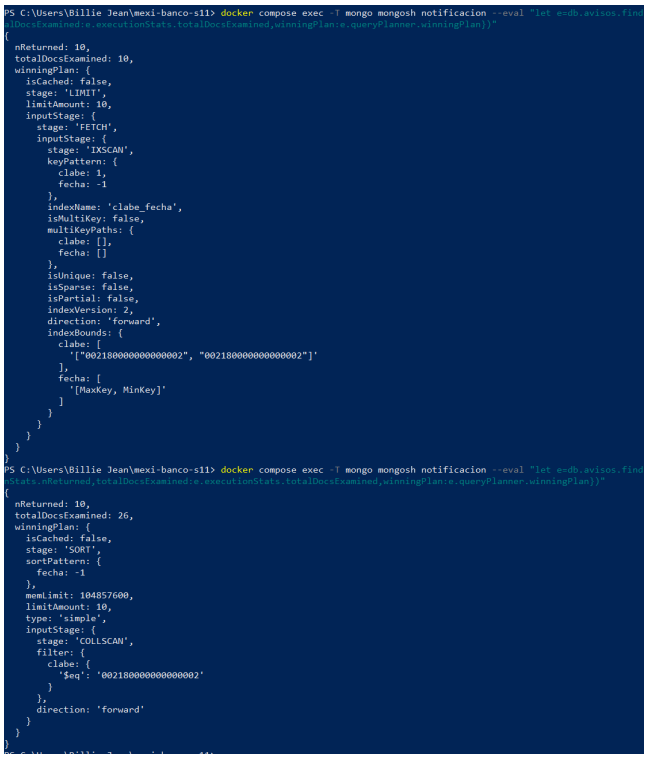


Figura 17: Comparación de IXSCAN y COLLSCAN mediante estadísticas de ejecución.

9. Prueba de idempotencia

9.1 Publicación de un evento duplicado

Se publicó dos veces un mismo evento de transferencia mediante la interfaz HTTP de RabbitMQ.

El evento utilizó dos CLABEs de prueba y un identificador fijo, para que ambas publicaciones representaran la misma operación.

RabbitMQ respondió `routed: true` en las dos publicaciones, confirmando que los mensajes fueron encaminados.

9.2 Verificación en MongoDB

Después de publicar el evento, se consultó la colección de avisos filtrando por su identificador.

El resultado fue de dos documentos, correspondientes a los dos destinatarios del evento, en lugar de cuatro.

Los logs de Notificación también mostraron dos mensajes `ya estaba`, indicando que se detectaron avisos existentes y no se duplicaron.

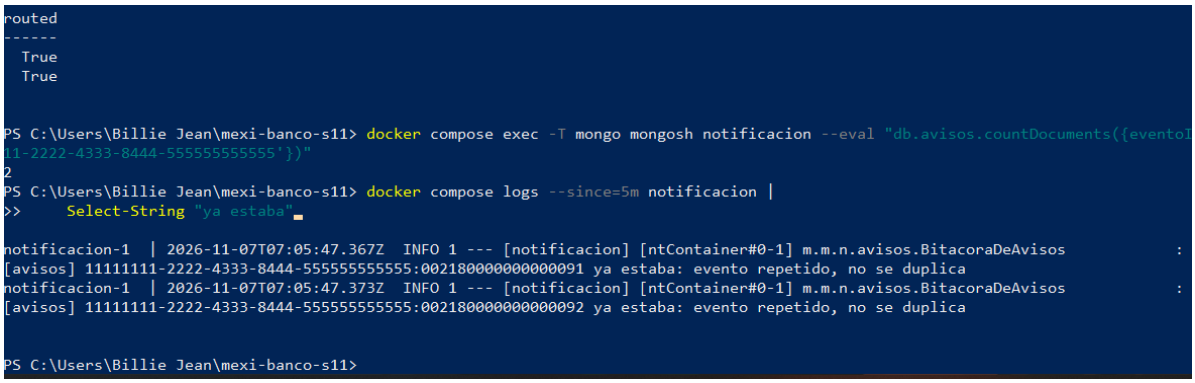


Figura 18: Evento publicado dos veces sin duplicar avisos en MongoDB.

> La prueba confirmó la idempotencia del registro de notificaciones: aunque un evento sea recibido más de una vez, se conserva un solo aviso por destinatario para ese identificador de evento.

10. Prueba de información desactualizada

10.1 Preparación del escenario

Se realizó una prueba para observar qué ocurre cuando Redis mantiene una copia antigua del saldo.

Primero se guardó el saldo original de Ana, que en ese momento era de $629.00.

Después se consultó la cuenta para almacenar el dato en Redis.

Posteriormente, se modificó temporalmente el saldo en PostgreSQL a $5.00 mediante una sentencia

SQL directa.

Este cambio se realizó únicamente como parte de la prueba controlada.

10.2 Diferencia entre Redis y PostgreSQL

Al consultar inmediatamente la API, el saldo mostrado continuó siendo de $629.00.

Sin embargo, PostgreSQL ya contenía el saldo de $5.00.

Esto demostró que Redis podía mantener temporalmente información desactualizada.

Después de que terminó el TTL, una nueva consulta mostró el saldo de $5.00.

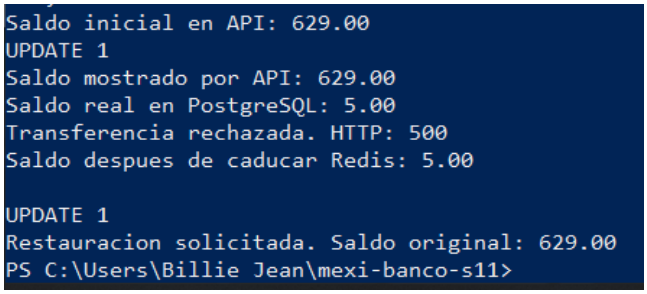


Figura 19: Redis muestra un saldo antiguo mientras PostgreSQL contiene el valor actualizado.

10.3 Intento de transferencia

Mientras PostgreSQL contenía solamente $5.00, se intentó realizar una transferencia de $100.00.

La operación no se completó correctamente y el servidor respondió con HTTP 500.

Sin embargo, al revisar los logs se encontró un error asociado a la restricción de PostgreSQL `transferencia_estado_check` , con código SQLState 23514.

También se observaron mensajes relacionados con `COMPENSACION_PENDIENTE` .

Por lo tanto, aunque la transferencia no terminó exitosamente, esta evidencia no permite afirmar que el rechazo se haya producido exclusivamente por saldo insuficiente.

10.4 Restauración del saldo

Al terminar la prueba se restauró el saldo original de Ana mediante PostgreSQL.

La actualización confirmó una fila modificada y se eliminó la entrada correspondiente de Redis.

Finalmente, se consultó tanto PostgreSQL como la API, obteniendo el mismo saldo de $629.00.



Figura 20: Saldo de Ana restaurado y registros del error observado.

La prueba confirmó que la información de Redis puede permanecer desactualizada hasta que expire su TTL.
También permitió identificar un problema adicional en el manejo de estados y compensaciones del servicio Transferencia.
El saldo original se restauró al finalizar el experimento.

11. Preguntas de cierre

MongoDB resulta conveniente para los avisos porque cada canal tiene información diferente. Por ejemplo, SMS utiliza un teléfono y un texto, CORREO contiene un destinatario, asunto y cuerpo, mientras que PUSH requiere un token, título y cuerpo. Además, los avisos se consultan principalmente por CLABE y fecha, como comprobamos al recuperar los últimos diez.

En cambio, el saldo necesita garantizar consistencia y transacciones confiables, ya que no se debe permitir que dos operaciones utilicen incorrectamente el mismo dinero. Por eso, PostgreSQL permanece como la fuente de verdad para las operaciones bancarias.

2. ¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?

En el escenario A obtuvimos un hit ratio de 99 %, con 99 aciertos y un fallo. En el escenario B disminuyó a 83.33 %, con 100 aciertos y 20 fallos. Esto ocurrió porque las transferencias modificaron los saldos e invalidaron las entradas de Redis.

Antes de activar la caché, las diez consultas realizadas generaron lecturas a PostgreSQL. Con Redis, en el escenario A, solamente se registró un fallo de caché durante las cien consultas.

Respecto a la latencia, el promedio disminuyó de 27.86 ms sin Redis a 14.28 ms con Redis, lo que representa una reducción del 48.7 %.

Por lo tanto, la caché aporta velocidad al usuario, pero también capacidad al sistema, ya que reduce la cantidad de consultas que PostgreSQL debe atender y permite aprovechar sus recursos para otras operaciones.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

Podría autorizarse una operación con información desactualizada. En nuestra prueba, Redis mostraba un saldo de $629.00, mientras que PostgreSQL registraba solamente $5.00.

Si se autorizara un cargo de $100.00 utilizando la copia de Redis, el sistema podría considerar que existe dinero suficiente cuando realmente no está disponible.

Durante nuestra prueba, la transferencia no se completó, aunque el servidor respondió HTTP 500 por un problema relacionado con una restricción de estados en PostgreSQL. Por ello, el resultado no permite confirmar que se haya rechazado correctamente por saldo insuficiente.

4. Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?

Se utiliza DEL para eliminar el dato anterior y permitir que la siguiente consulta recupere el saldo confirmado desde PostgreSQL.

Hacer SET directamente podría guardar un valor que todavía no corresponde al estado definitivo de la operación, especialmente si ocurre un error o existen movimientos concurrentes.

La eliminación debe realizarse después del commit porque, si se hace antes, otra consulta podría leer el saldo antiguo de PostgreSQL y volver a guardarlo en Redis.